An Energy-Efficient POSIT Compute-in-Memory Macro for High-Accuracy AI Applications

Yang Wang, Xiaolong Yang, Yubin Qin, Zhiren Zhao, Ruiqi Guo, Zhiheng Yue, Huiming Han, Shaojun Wei, *Fellow, IEEE*, Yang Hu, *Member, IEEE*, and Shouyi Yin, *Fellow, IEEE*

***Abstract*— Floating-point (FP) CIM is an attractive technique that achieves high accuracy with considerable energy efficiency, especially for complex artificial intelligence tasks. However, FP-CIM with FP32/FP16/BF16 data format incurs a performance bottleneck due to the large storage requirements and MAC power. The emerging POSIT data format exploits dynamic bit width that adapts to varied data distributions, enabling using a low-bit-width to reach nearly the same accuracy performance as conventional high-bit-width FP (POSIT8 ≈ FP16/BF16). Therefore, a POSIT-based CIM exhibits inherent advantages over the traditional FP-CIM. Despite the theoretical superiority, the dynamic property of POSIT introduces huge exponent processing overhead, cell underutilization of the CIM array, and redundant logic toggles in mantissa addition for POSIT-base CIM. This article presents a POSIT-based digital CIM (PD-CIM) macro with three features to tackle the above challenges: 1) a bi-directional regime processing unit (BRPU) simplifies the complicated codec logic to reduce the energy overhead in dynamic exponent processing; 2) a critical-bit pre-compute-and-store (CPCS) CIM utilizes the spare CIM cells caused by the dynamic bit-width of mantissa to improve cell utilization of the CIM array; and 3) a cyclically alternating computing-scheduling unit replaces the bit-wise addition of dynamically aligned mantissas with bit-wise OR to save logic toggles power. Fabricated in a 28 nm CMOS technology, PD-CIM occupies an area of 1.41 mm^2. It reaches an 83.23 TFLOPS/W peak energy efficiency at POSIT(8,1) for ImageNet classification on the vision transformer base (ViT-B) model. Compared with the state-of-the-art FP-CIM, PD-CIM reduces energy by 2.36× and offers 3.51× speedup.**

***Index Terms*— Artificial intelligence (AI), compute-in-memory, floating-point (FP), high-accuracy, POSIT data format.**

Received 6 June 2024; revised 15 October 2024; accepted 18 January 2025. Date of publication 5 February 2025; date of current version 25 July 2025. This article was approved by Associate Editor Jie Gu. This work was supported in part by NSFC under Grant 62304121, Grant 62125403, Grant U19B2041, and Grant 92164301; in part by the National Key Research and Development Program under Grant 2021ZD0114400; in part by the Project supported by Shanghai Municipal Science and Technology Major Project; in part by Beijing National Research Center for Information Science and Technology; and in part by Beijing Advanced Innovation Center for Integrated Circuits. *(Corresponding author: Shouyi Yin.)*

Yang Wang, Xiaolong Yang, Yubin Qin, Zhiren Zhao, Ruiqi Guo, Zhiheng Yue, Huiming Han, Shaojun Wei, and Yang Hu are with the School of Integrated Circuits, Beijing Innovation Center for Future Chip, and Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China.

Shouyi Yin is with the School of Integrated Circuits, Beijing Innovation Center for Future Chip, and Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China, and also with Shanghai AI Lab, and the International Innovation Center of Tsinghua University, Shanghai 200062, China (e-mail: yinsy@tsinghua.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSSC.2025.3532654.

Digital Object Identifier 10.1109/JSSC.2025.3532654

I. Introduction

RECENTLY, artificial intelligence (AI) models have undergone rapid advancements, enabling handle more intelligent tasks such as autonomous driving, 3-D reconstruction, and video generation [1], [2], [3], [4]. The high intelligence of AI models relies on the growth in model scale and high-precision computations. On one hand, the scale of AI models has surged by over 10 000× in the past decade. As an illustration, ResNet152, proposed in 2015, boasted 6 million parameters. Contrasting this, the contemporary AI model Llama2 flaunts a staggering 70 billion parameters, indicating 11 667× more data movement energy than ResNet152. On the other hand, compared to INT8 used for conventional AI models, the emerging AI applications require high-precision floating-point (FP) computations to maintain accuracy [5], [6], [7], [8], [9], [10], [11], [12], [13], [14]. For the inference task of image generation on the diffusion model, INT8 degrades accuracy by over 10% than FP16. Compared to INT8, FP16 introduces a bottleneck since it consumes 6.25× more energy and leads to a 1.5× longer critical path. In addition, on-chip training has become an indispensable requirement to adapt to diverse and private applications [8], [9], [10], [11], [12], [13], also demanding FP operations to ensure accuracy. These two factors inspire an urgent need for designing an energy-efficient AI processor with high-precision computing.

FP-CIM is a promising technology that enhances the energy efficiency of AI processors by avoiding data movement while also ensuring accuracy for inference of complex AI tasks and training [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26]. However, FP-CIM with conventional FP data formats, such as FP32/FP16/BF16, encounters a performance bottleneck. Taking a specific training iteration of the ViT-B as an example, the gradient, activation, and weight exhibit varied data distributions, as shown in Fig. 1. FP-CIM requires high bit-width data formats, such as FP16/BF16, to simultaneously cover the data range of gradients, activations, and weights while ensuring computational accuracy. The high bit-width FP data significantly increases energy consumption, thus limiting the energy efficiency of FP-CIM.

Different from the conventional FP with a fixed format, the emerging POSIT data format enables dynamic bit allocation to achieve range and precision trade-off at runtime [27], [28], [29], [30], [31], [32], [33]. A POSIT data consists of four parts: sign (S), regime (R), exponent (E), and mantissa (M). S is the

0018-9200 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

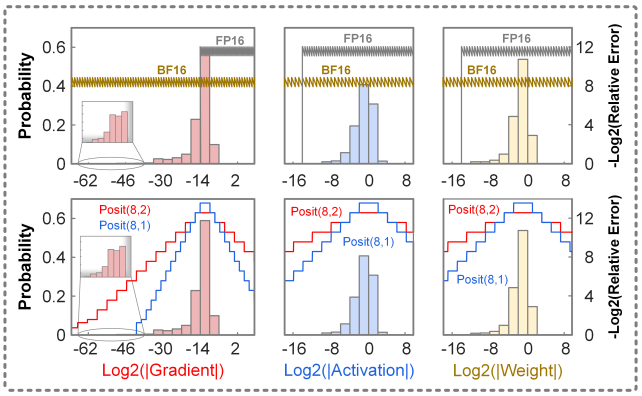


Fig. 1. Relative error comparisons of different data formats.

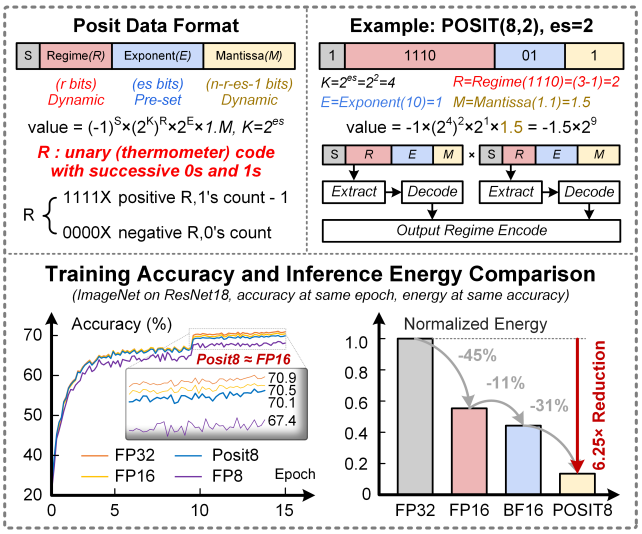


Fig. 2. Principle and performance of POSIT data format.

1-bit sign flag. It is "0" for a positive POSIT number, and it is "1" for negative numbers. It can dynamically adjust the bit widths of R, E, and M to adapt to varied data distributions, as shown in Fig. 1. As a result, low-bit-width POSIT data (POSIT8) can achieve nearly the same accuracy as the high-bit-width FP data (FP16), significantly reducing the memory access and MAC energy.

Despite the theoretical superiority of the POSIT data format, its dynamic data characteristics introduce three challenges for applying POSIT to CIM. First, the dynamic regime processing of POSIT data increases computing power by 2.62×. Second, the CIM array suffers from 41.3% of CIM cells underutilization when storing the dynamic bit width mantissa of the POSIT data. Third, during the mantissa accumulation of the POSIT data, the adder tree incurs 62.5% of energy waste.

This article proposes a POSIT-based digital CIM (PD-CIM) macro [22], addressing the challenges caused by the dynamic property of the POSIT data format. The proposed CIM macro achieves a peak energy efficiency of 83.23 TFLOPS/W. For the ImageNet classification on ViT-B, the PD-CIM reduces energy by 2.36× and offers 3.01× speedup compared with the state-of-the-art FP-CIM macro [15]. The main innovations of the proposed processor are as follows.

1) A bi-directional regime processing unit (BRPU) simplifies the complicated codec logic in the regime processing with shifting and concatenation by using the successive 0/1 thermometer coding property of the regime, saving 40.3% of regime processing energy.
2) A critical-bit pre-compute-and-store (CPCS) CIM array utilizes the spare CIM cells to store pre-computed partial summation result, which facilitates the CIM array to perform dual-bit MACs in each cycle, improving 63.0% of the utilization for the CIM array.
3) A cyclical-alternating compute scheduling unit (CASU) uses energy-efficient bit-wise OR logic operations to replace data-wise addition if the two dynamically aligned mantissas have no overlap bits, reducing the computing energy of the adder tree by 56.9%.

The rest of this article is organized as follows. Section II introduces the basic principles of the POSIT data format and the challenges of designing POSIT-based CIM macro. Section III presents the overall architecture of the proposed POSIT-based CIM macro. Then, Sections IV–VI detail the three innovations. Finally, Section VII presents the measurement results, and Section VIII concludes this article.

II. Background and Motivation

A. Principles of POSIT Data Format

Over the past decade, AI models have developed rapidly and become increasingly intelligent [1], [2], [3], [4]. The dramatic growth in model parameters and high-precision computations are the two crucial factors ensuring a high intelligence level in AI models. To reduce data movement energy consumption and ensure computational accuracy, FP-CIM has emerged [15], [16], [17], [18], [19], [20], [21], [22], [23]. However, FP-CIM with traditional fixed-format FP data cannot adapt to diverse data distributions, leading to performance bottlenecks. As shown in Fig. 1, during a training iteration of the ViT-B, the exponent ranges for gradients, activations, and weights are $[-66, -10]$, $[-16, 4]$, and $[-16, 2]$, respectively. For the FP8 data with (1, 5, 2) format, the exponent representation range is only $[-14, 15]$. As we can see, although FP8 can effectively represent activations and weights, it cannot cover the exponent range of gradient. Therefore, all gradients with exponents less than -14, which account for 69.3% of the data, are forced to be represented as 0 in the FP8 data format, resulting in a loss of training accuracy. Therefore, higher bit-width FP formats such as FP16 or BF16 are necessary to ensure training accuracy. Nevertheless, the memory access energy consumption and MAC energy consumption of the FP16 data format are 2× and 3.06× that of FP8, respectively, limiting the energy efficiency of FP-CIM.

A promising solution to achieve range and precision trade-off is the emerging POSIT data format [27], [28], [29], [30], [31], [32], [33], consisting of (S, R, E, M), and R, E, and M have dynamic bit-widths determined by the varied data distributions. Generally, the n-bit POSIT data have a format of POSIT(n, es), where es is the bit width of E. The decimal value of a POSIT(n, es) data is $(-1)^S \times (2^{es})^R \times 2^E \times 1 \cdot M$. As we can see, a large es indicates a wide exponent

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

range, while a small es denotes a high mantissa precision. The bit widths of R and M vary dynamically, allowing a data range and precision trade-off at runtime. Assuming the bit widths of R and M are r and m, respectively, then r and m will have the relationship of $m = n - r - \text{es} - 1$. R in POSIT data is a unary (thermometer) code with successive 0s and 1s, e.g., 11110 and 00001, as shown in Fig. 2. The decimal value of R is the number of successive 1's count minus a constant 1 for positive values, and it is successive 0's count for the negative values ($R = 3$ for "11110," $R = -4$ for "00001"). The thermometer code can automatically segment the bits of the POSIT data, allowing us to dynamically distinguish regime and mantissa with leading one or leading zero detectors, which decode R to obtain its decimal value for MAC computing. With the dynamic bits allocation, the POSIT8 data can achieve less than 0.4% accuracy loss compared to conventional FP16 or BF16 and reduce the energy by 6.25× for image classification on the ResNet18, as shown in Fig. 2.

B. Multi-Scope Challenges Caused by Similar Vectors

Nevertheless, despite the theoretical superiorities of POSIT data format, its dynamic property introduces three challenges for designing an energy-efficient POSIT-based CIM macro, as illustrated in Fig. 3.

First, the dynamic regime processing of POSIT-based computing introduces an energy bottleneck. As shown in Fig. 3, unlike the fixed bit-width exponent processing in the conventional FP data, the regime processing of POSIT data requires additional extract and codec logic. During the MAC operation with two POSIT operands, the computing unit first requires extracting the thermometer-code-based dynamic regime with leading one or zero detectors. Then, it decodes the thermometer code to obtain the decimal value for MAC calculation and encodes the output result into the thermometer format regime. The extract and codec processing logic increases the energy by 2.62× than exponent processing in conventional FP, limiting the energy efficiency of POSIT-based CIM.

Second, the dynamic bit-width mantissa of the POSIT data incurs CIM cell under-utilization, limiting the throughput of the CIM array. Generally, a CIM array may have a pre-defined structured architecture based on the fixed bit-width mantissa to achieve 100% utilization, as shown in Fig. 3. However, the mantissas of POSIT data have varied bit-width based on the regime for exponent range and mantissa precision trade-off. A POSIT operand with a large bit-width regime indicates that we lean toward increasing its exponent range but sacrifice its mantissa precision, or vice versa. The dynamic bit-width mantissa conflicts with the structured CIM architecture, resulting in a 41.3% CIM cell under-utilization.

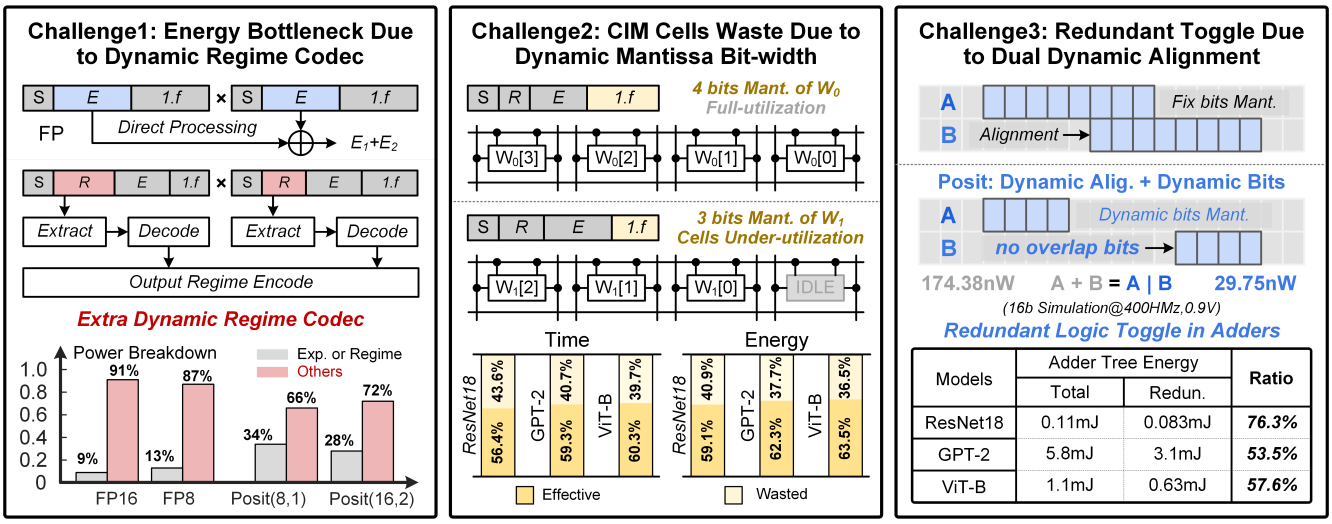
Third, the adder tree with the dynamically aligned mantissa of POSIT data suffers from redundant logic toggles. For the adder tree, once the dynamically aligned two mantissas have no overlap bits, all the full adders perform X + 0 = X operations. This situation is prevalent in the POSIT data, whose mantissa bit-widths are smaller than conventional FP16 or BF16. In this case, the computing result of X + 0 equals X|0, as shown in Fig. 3. It denotes redundant logic toggles inside the adders. According to the simulation results on a 16-bit adder and a 16-bit OR logic at 400 MHz with TSMC 28 nm technology, the power consumptions are 174.38 and 29.75 nW, respectively. Thus, compared with the OR-based accumulation, the adder tree wastes 62.5% of computing energy.

III. Overall Architecture

Fig. 4 presents the overall architecture of the proposed PD-CIM macro, consisting of a BRPU, 16 CIM cores with CPCS CIM arrays, 16 adder trees with CASU, a 28 kB global SRAM, an FP-to-POSIT encoder, a POSIT-to-FP decoder, and a top-controller. Similar to the previous FP-CIM [15], [16], [17], PD-CIM separately stores the regime in the global SRAM for digital processing and stores the mantissas with dynamic bit-width in the CIM array for MAC computations. Each CIM core contains 16 CIM arrays. Each CIM array has an 8 × 48 CIM sub-array, where every 4b CIM cell is a CIM unit. When loading weight data to PD-CIM macro, the FP-to-POSIT encoder transfers the FP data to POSIT format for storage. Along with the data loading, the CPCS detects the weight mantissa bit width to pre-compute and stores a logical value in the spare CIM cells. After that, during computing, BRPU performs shift-OR-based regime processing. The CIM array with CPCS has three work modes, including 3b mode, 2-4b set-associate mode, and normal mode. For the former two modes, the CIM array enables using the pre-computed and stored logical value to achieve inter-CIM dual-bit MAC in each cycle. The CIM array performs single-bit-serial computing when working in the normal mode. Finally, CASU receives the computing results from the CIM array for accumulation. It can save energy of the adder tree by substituting the addition to bit-wise or operation for the no overlap mantissas. Besides, it changes the computing order of the CIM array to increase the number of no-overlap mantissas.

In practical applications, PD-CIM functions as a computing engine to accelerate neural processing. It requires to work in conjunction with a host processor to complete computations. For a specific application scenario, the host processor needs to analyze the distribution characteristics of the input operands, such as activation, weight, and gradient. Based on the distribution, the host processor selects a specific POSIT data format, including POSIT(16, 2)/POSIT(16, 3)/POSIT(8, 1)/POSIT(8, 2), for computing. PD-CIM achieves POSIT16 computing by fusing multiple regime processing units, CIM units, and adder units of POSIT8. On average, the hardware logic consumed for each POSIT16 computation is twice that of POSIT8. According to our inference and training experimental results, these four options can meet the requirements of typical AI applications, such as ImageNet classification inference on ResNet152 and natural language processing training on GPT-2. After determining the POSIT data format, the FP-to-POSIT encoder in PD-CIM converts conventional FP data to the selected POSIT format for computing, and the POSIT-to-FP decoder recovers the computed results to standard FP data for the system. In this way, the proposed PD-CIM macro is compatible with existing AI processors.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.



Models	Adder Tree Energy		Ratio
	Total	Redun.	
ResNet18	0.11mJ	0.083mJ	*76.3%*
GPT-2	5.8mJ	3.1mJ	*53.5%*
ViT-B	1.1mJ	0.63mJ	*57.6%*

Fig. 3. Three challenges of designing energy-efficient POSIT-based CIM caused by the dynamic properties of POSIT data format.

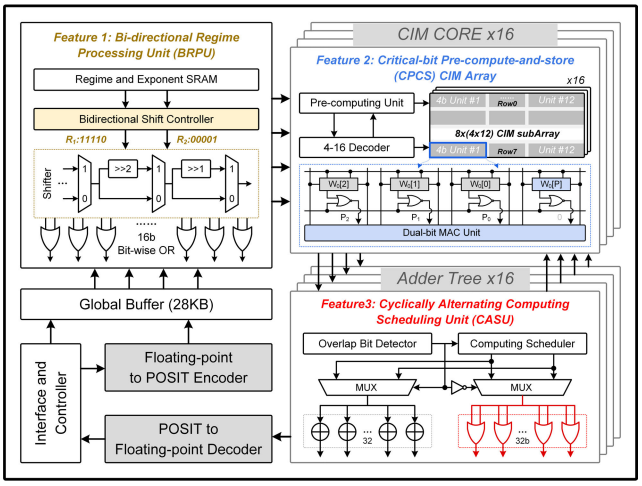


Fig. 4. Overall architecture of the proposed PD-CIM.

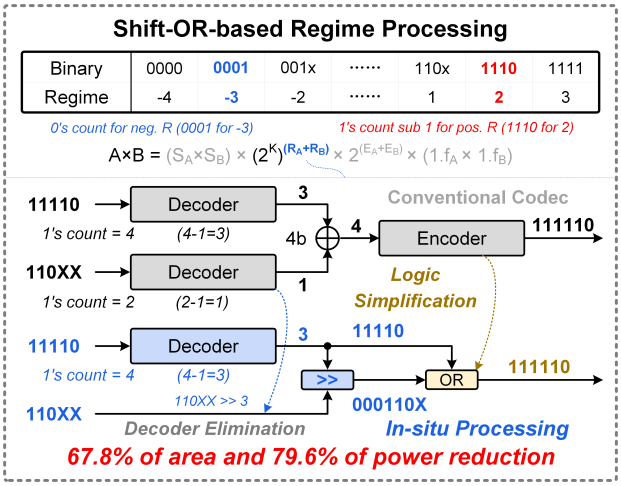


Fig. 5. Proposed shift-OR-based regime processing.

IV. Bi-Directional Regime Processing Unit

POSIT data utilize a regime, which is a thermometer code with dynamic bit-width for data range and precision trade-off [27], [28], [29]. The thermometer code is successive "0"/"1" followed by "1"/"0" to achieve automatic bit partition. POSIT-based computing requires extracting the regime and decoding the thermometer codes to the decimal values for processing, resulting in an energy bottleneck. We propose a BRPU that simplifies the complicated regime processing logic to the shift-OR-based operations by using the regime's successive "0"/"1" coding property, significantly reducing the overhead.

A. Shift-OR-Based Regime Processing

POSIT data use a thermometer code regime with successive "0"("1") followed by "1"("0") to represent a negative (positive) decimal value. Specifically, a regime of "0001" denotes a decimal value of -3 as "0001" has three successive "0s." That is, the leading one position of "0001" can indicate the decimal value. During the POSIT-based multiplication of $A \times B$, PD-CIM needs to extract the regimes of A/B (R_A/R_B) and use leading one/zero detectors to obtain their decimal values (R_{DA}/R_{DB}). After that, it computes $R_{DA} + R_{DB}$ and encodes the addition result to get the output regime. As shown in Fig. 5, this process needs two decoders, one adder, and one encoder, which are area and power-consuming.

For R_A/R_B using the count of successive "0"/"1" to represent decimal values, the computing of $R_{DA} + R_{DB}$ signifies an increase or decrease in the count of the successive "0"/"1." Based on this property, we propose a BRPU, which decodes R_A to a decimal R_{DA}, shifts the R_B for R_{DA} bits to get R'_B, and concatenates R_A with R'_B through a bit-wise OR logic. This shift-OR-based operation obtains the same output regime as the conventional computing. It saves one decoder and simplifies the original adder/encoder to the shift-OR logic, saving 67.8% of the area and 79.6% of the power than the conventional regime processing. In addition, it reduces 23.6% of the area and 22.2% of the power for the CIM macro.

Despite the simplicity of the shift-OR operation, randomly selecting one of R_A/R_B for decoding while shifting the other

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

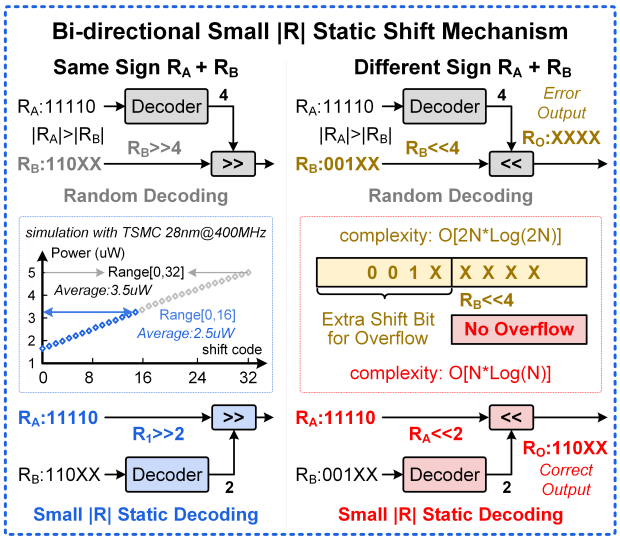


Fig. 6. Bi-directional small $|R|$ static shift mechanism.

operand leads to energy waste and shift errors. As shown in Fig. 6, for R_A/R_B with the same sign, $R_{\mathrm{DA}} + R_{\mathrm{DB}}$ indicates right shift to increases the count of successive "0"/"1." In this case, if $|R_A| > |R_B|$ and we randomly decode R_A while shifting R_B, the shifter consumes a significant amount of energy, as the power consumption of the shifter increases linearly with the shift code. Besides, for R_A/R_B with different signs, $R_{\mathrm{DA}} + R_{\mathrm{DB}}$ denotes left shift to decrease the count of successive "0"/"1." Randomly decoding R_A and shifting R_B incurs shifting errors once $|R_A| > |R_B|$. For example, assuming the decoded value of R_A is 4 and R_B is "001XX," left shifting "001XX" by 4 bits results in a "XXXXX," which is an error output. Adding extra shift bits can solve the shift overflow issue, but it will double the cost of the shifter. We propose a bidirectional shift controller, which dynamically decodes the small $|R|$ while shifting the large $|R|$ to save energy and to avoid errors. For R_A/R_B with the same sign, it reduces the shift energy since a smaller shift code indicates less shift power. In addition, for R_A/R_B with different signs, it avoids shift errors without using extra bits, as shown in Fig. 6.

B. BRPU for Shift-OR-Based Regime Processing

Fig. 7 illustrates the overall architecture of BRPU, encompassing a regime fetch unit, a bi-directional shift controller, a digital control shift line, and a 16-bit output regime register file. We detail the workflow of BRPU with two POSIT(8, 3) operands. In each cycle, the fetch unit gets 2×4 bits R_A/R_B for processing, where 4 is the maximum bit-width of regime for the POSIT(8, 3) operands. The bi-directional shift controller receives the thermometer code-based R_A/R_B and dynamically decodes the small $|R|$ while shifting the large one. Instead of decoding the thermometer codes of R_A/R_B to decimal values for comparison, the bi-directional shift controller catches the smaller $|R|$ still with the thermometer code. Taking R_A/R_B with the different signs as an example, it first performs 4-bit XOR with R_A/R_B to obtain an output. Then, a 4-bit leading-zero detector located the position of the leading zero bit, which denotes the value of the small $|R|$. The theory behind this operation is that a small $|R|$ will have less successive "0"/"1," indicating early $1 \rightarrow 0$ change. Similarly, the bi-directional shift controller detects the smaller $|R|$ for R_A/R_B with the same sign via 4-bit XOR and a 4-bit leading-one detector.

During the bi-directional shift scheduling, a merged leading one/zero detector saves the overhead of capturing small $|R|$. As shown in Fig. 5, the regime of POSIT utilizes successive "0" followed by "1" to represent a negative decimal value. On the contrary, it denotes a positive decimal value with "1" followed by "0." Theoretically, to distinguish between positive and negative regimes, we need a leading one detector and a leading zero detector, followed by a 2-to-1 MUX to obtain the corresponding decimal value, as shown in Fig. 7. When simulated on a 4-bit regime detection with TSMC 28 nm technology at 400 MHz, this separated detection approach consumes an area of 4.93 μm^2 and a power of 0.31 μW, increasing the overhead of regime processing. Instead of using the separated detection, we propose a merged leading one/zero detector. As shown in Fig. 7, we integrate the truth tables of leading one detection and leading zero detection to obtain a unified logical expression, allowing us to reuse several logic circuits between them and reduce overhead. Under the same simulation setup, the proposed merged approach saves area by 28.4% and power by 25.8% than the separated implementation.

V. CPCS CIM Array

Unlike conventional FP-CIM, which uses a structured CIM array to store mantissas with fixed bit-width, PD-CIM requires storing dynamic bit-width mantissas with the CIM array. Due to the random distribution of the dynamic bit-width mantissas, the CIM array with pre-defined structured architecture suffers from CIM cell under-utilization. We propose a CPCS CIM array that utilizes the spare CIM cells to store the pre-computed partial summation results. It can facilitate the CIM array to complete two multiplications in each cycle, improving the CIM cell utilization of the CIM array.

A. CPCS Mechanism

Generally, FP-CIMs use a CIM array to store the mantissas for multiplication [15], [16], [17], [18], [19], [20], [21], [22], [23]. For conventional FP data with fixed bit-width mantissa, we can easily design a structured CIM array where each CIM unit has the same bit-width as the mantissa to achieve 100% CIM cell utilization. However, for POSIT data format, the bit-width of mantissa requires dynamically varied to adapt to different data distributions. It introduces CIM cells under-utilization for the CIM array with a fixed structure. Fig. 8 shows the proportion of weights with different bit-width mantissas during the training of ResNet18 based on POSIT(8, 1) and POSIT(8, 2). The mantissas with 2/3/4-bit widths account for the majority of the proportion. Assuming we pre-set the CIM unit as 4-bit, the CIM unit incurs CIM cell waste for storing 2/3-bit mantissas, which takes a proportion of 61.9% for POSIT(8, 1). The situation for POSIT16 format is more complex since the mantissa bit-width of POSIT16 data

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

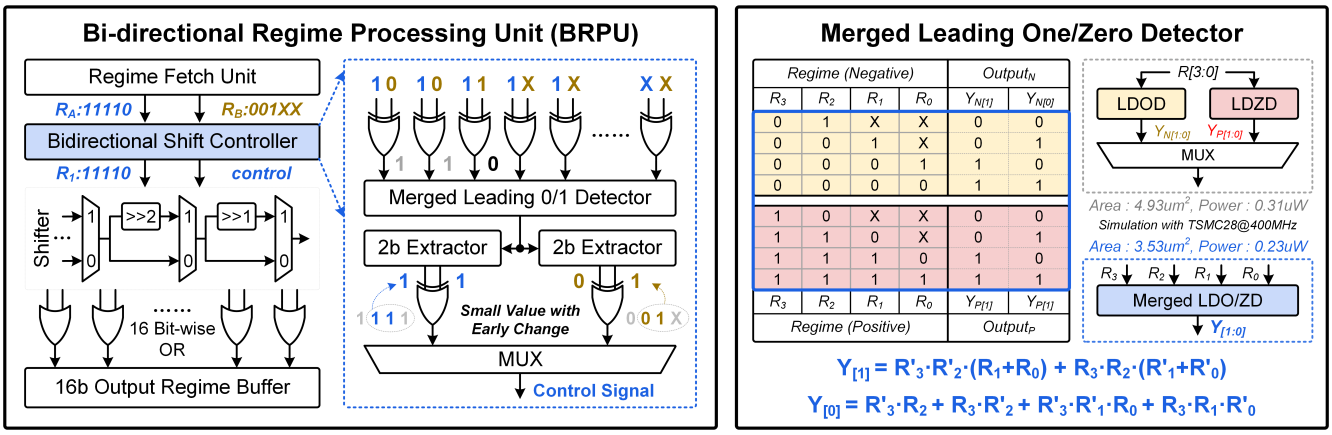


Fig. 7. Architecture and design details of BRPU.

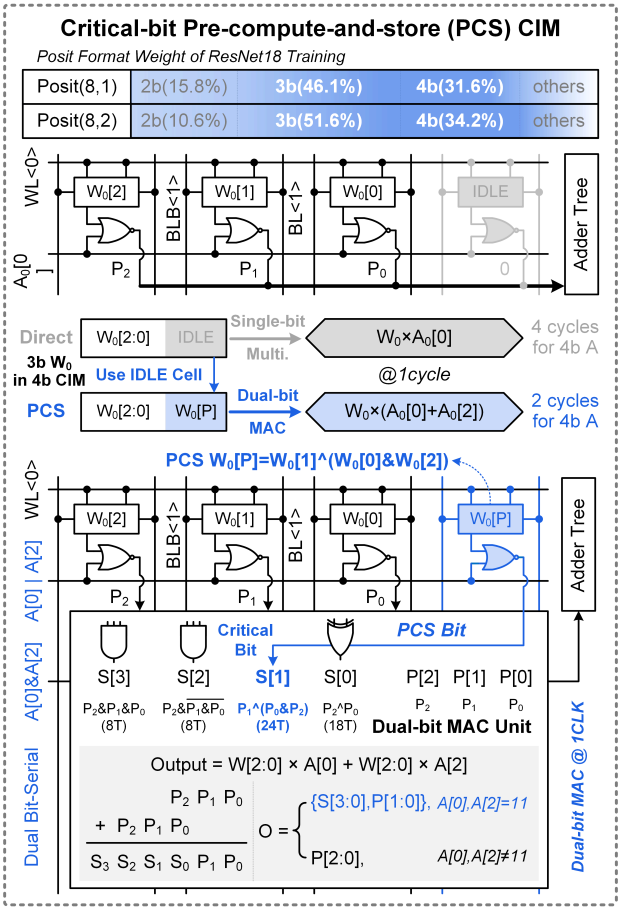


Fig. 8. Principle of CPCS mechanism.

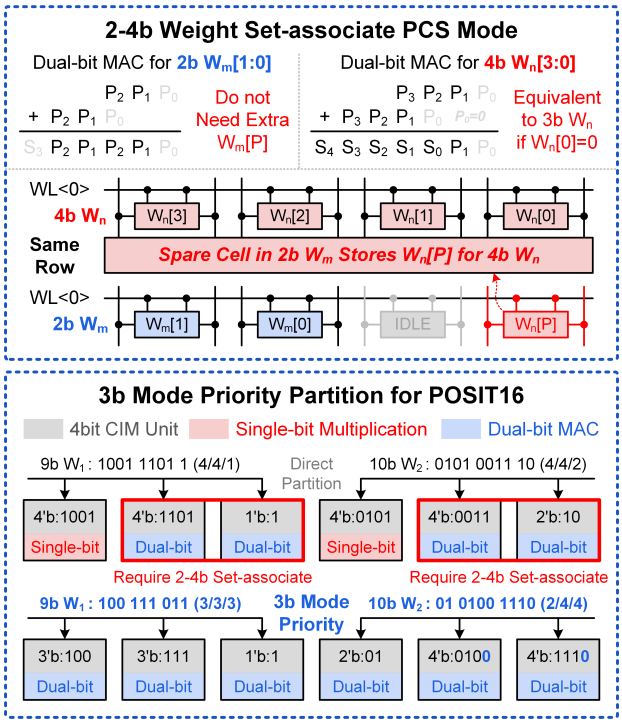


Fig. 9. CPCS with 2-4b weight association and 3b priority partition.

is in the range of [0:12]. If pre-setting the CIM unit as 12-bit, the waste ratio of POSIT16 is as large as 85.7%. A direct solution is to design a CIM array with dynamic bit-widths, but it is impracticable as the N-bit CIM cells with dynamic partition introduces a routing complexity of $O(N^2)$.

We propose a pre-compute-and-store mechanism that maintains the structured architecture of the CIM array while effectively utilizing the spare CIM cells to improve throughput. Fig. 8 illustrates the principle of the proposed pre-compute-and-store approach using the example of storing a 3-bit weight mantissa into the 4-bit CIM unit. Assuming the stored 3-bit weight mantissa is W, unlike the conventional single-bit-serial computing, the 4-bit CIM unit computes dual-bit MACs of $O = W \times A[n] + W \times A[n+2]$ in each cycle. Specifically, the CIM unit receives $A[n]|A[n+2]$ and performs the multiplication of $W \times (A[n]|A[n+2])$ to obtain a partial product of $P[2:0]$. It then obtains O with $P[2:0]$ according to $A[n+2]/A[n]$. If $A[n+2]/A[n] = 00$ or 01, then $O = P[2:0]$. Else, if $A[n+2]/A[n] = 10$, $O = P[2:0] \ll 2$. Otherwise, $O = P[2:0] + P[2:0] \ll 2 = [0, 0, P_2, P_1, P_0] + [P_2, P_1, P_0, 0, 0]$. The CIM unit can get O by computing 3-bit $S[3:0] = [0, 0, P_2] + [P_2, P_1, P0]$ and concatenates $[P_1, P_0]$. The computation of $S[1]$ consumes the largest number of transistors, defined as the critical bit, where $S[1] = W[P] =$

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

$W[1]\hat{}(W[0] \ \& \ W[2])$. Rather than computing $S[1]$ at runtime, CPCS pre-computes $S[1]$ and stores it in the spare CIM cell, facilitating the CIM unit completing dual-bits MACs ($W \times A[n] + W \times A[n+2]$) in each cycle. Theoretically, for 4-bit mantissas stored in the 4-bit CIM unit, we have to perform single-bit-serial computing since there is no spare cell to store the pre-computed critical bit, limiting the throughput. We propose a 2–4 bit association method to enable dual-bit MACs for the 4-bit mantissas. As shown in Fig. 9, once the LSB of the stored 4-bit W_n is "0," which means $W_n[0] = 0$, we can regard the $W_n[3:1]$ as a 3-bit weight and compute $W_n[3:1] \times A[n] + W_n[3:1] \times A[n+2]$ in each cycle. Besides, since the 2-bit $W_m[1:0]$ does not require extra $W_m[P]$ for dual-bit MACs, the 4-bit CIM unit storing 2-bit mantissa has two spare cells. Therefore, the 4-bit CIM unit storing 4-bit W_n can borrow a spare cell from the CIM unit with 2-bit W_m to pre-compute-and-store $W_n[P]$, as shown in Fig. 9. This 2–4 bit association increases the throughput by 31%.

In addition, we present a 3-bit priority partition method to improve the throughput of POSIT16 data. As shown in Fig. 9, since POSIT16 data have mantissas exceeding 4-bit, we need to use multiple 4-bit CIM units to store a single mantissa. A straightforward method is to preferentially use the former CIM units and put all spare cells in the last CIM unit. For example, this method separates a 9-bit mantissa of "100111011" into three segments: "1001," "1101," and "1XXX," where "X" indicates a spare cell. In this case, the CIM units for "1001" and "1101" cannot perform dual-bit MACs. Although we can associate the second segment of "1101" with the third segment of "1XXX" to utilize the spare cells in "1XXX" for dual-bit MAC operations of "1101," the first segment of "1001" still has to perform single-bit-serial computing. It limits the throughput since the three CIM units require synchronous computing. Unlike sequential partition, we propose a 3-bit priority partition, which separates "100111011" to "100X," "111X," and "011X." In this way, all the three CIM units can perform dual-bit MACs. Another case is we cannot separate the mantissa into several 3-bit segments, as illustrated by the 10-bit mantissa in Fig. 9. For this case, we dynamically adjust the segmentation to enable the LSB of the partitioned 4-bit segment to be "0." The 3-bit priority partition increases the throughput of POSIT16 computing by 1.34×.

B. CPCS CIM Array for Dual-Bit MACs Computing

Fig. 10 shows the overall architecture of the CPCS CIM array, consisting of an 8 × 48 CIM array, a load controller, a critical-bit computing unit, and a 3-to-8 decoder. The 48b cell of each row is separated into 12 CIM units, each containing 4 CIM cells. The 4b CIM unit has three work modes determined by the stored mantissa.

1) When storing a weight with 3-bit mantissa of W, the CIM unit receives $A[n]|A[n+2]$ in each cycle and performs dual-bit MACs of $W \times A[n] + W \times A[n+2]$.
2) When storing a 4-bit W with $W[0] = 0$, the CIM unit works with 2–4 bit association mode. It performs dual-bit MACs with $W[3:1]$ by utilizing the spare cell from another CIM unit storing 2-bit weight and left shifts the computed results by 1 bit for compensation.
3) When storing a 4-bit W with $W[0] = 1$, the CIM unit performs standard bit-serial computing. The load controller, critical-bit computing unit, and 3:8 decoder only work during weight loading or updating. Their overhead is only 1.6% since the CIM array works for computing for 98.7% of the cycles. During weight loading, the pre-computing unit obtains the critical bit of $W[P]$ for 3-bit W and 4-bit W with $W[0] = 0$. For a CIM unit storing a 4-bit W with $W[0] = 0$, if there is a CIM unit storing a 2-bit weight in the same row, the 3-to-8 decoder, whose 8-bit outputs connect to eight transmission gates, generates a 8-bit code to associate two CIM units with 2-bit and 4-bit mantissas, respectively.

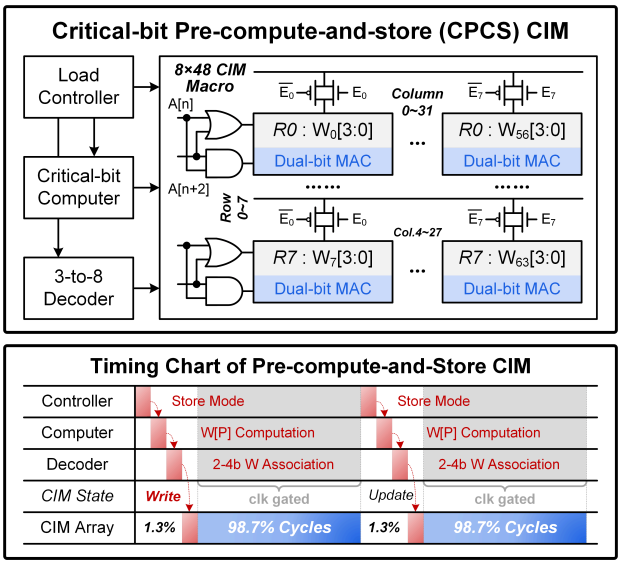


Fig. 10. Architecture and timing chart of CPCS CIM array.

VI. Cyclical Alternating Scheduling Unit

After the regime processing and mantissa multiplication, the adder tree dynamically aligns the multiplication results based on the computed regime for addition. In this process, the adder tree incurs energy waste since the dynamically aligned results from the CIM array may have no overlap bit, whose addition equals to the OR operation. It indicates that the logic toggles in the corresponding adders are redundant. We propose CASU that dynamically substitutes the data-wise addition to energy-efficient bit-wise OR based on the overlap conditions of two aligned multiplication results. It remarkably saves the energy of the adder tree for POSIT computing.

A. OR-Based Hybrid Computing and Cyclical Scheduling

A unique property of POSIT computing is that the dynamically aligned results are more likely to have fewer overlap bits than conventional FP computing due to the less mantissa bit-width of POSIT data. Specifically, for FP8 (1, 4, 3), the mantissa multiplication of two FP8 operands results in an 8-bit

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

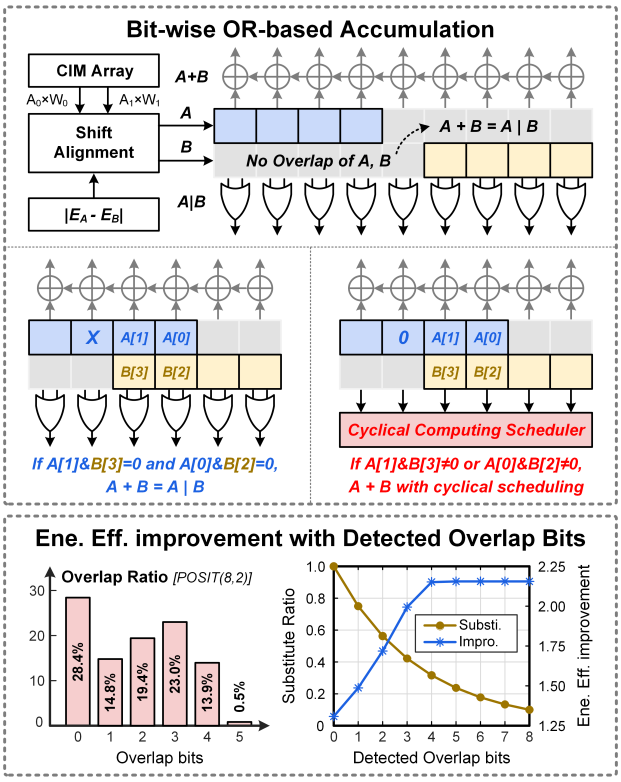


Fig. 11. OR-based accumulation for mantissas with no overlap bits.

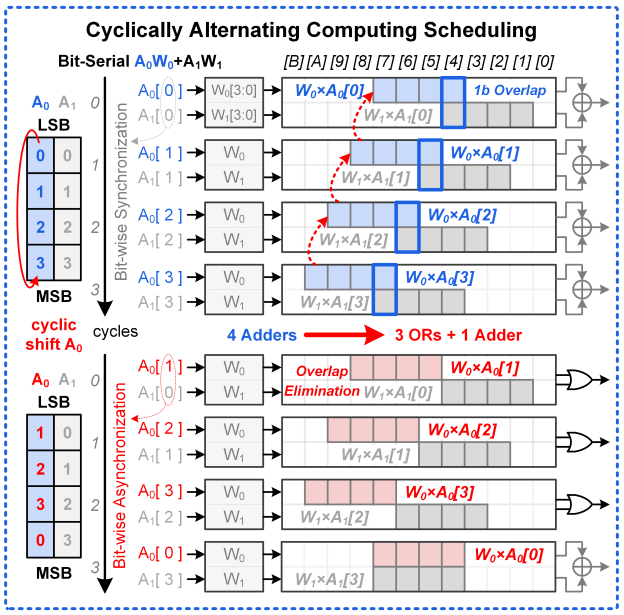


Fig. 12. OR-based accumulation with cyclical alternating scheduling.

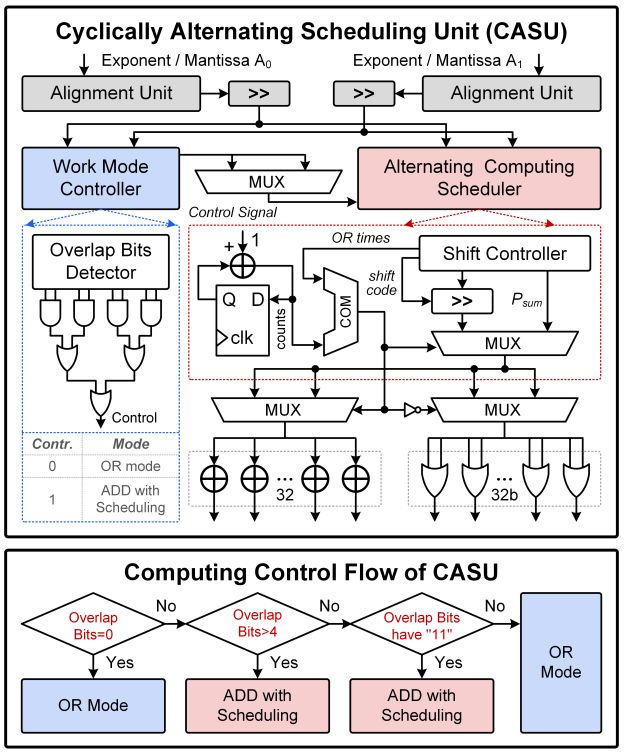


Fig. 13. Architecture and computing control flow of CASU.

output, which is always less than 6-bit for POSIT8 since 61.9% of POSIT8 data's mantissa bit-width is smaller than 3-bit, as shown in Fig. 8. Thus, for the identical exponent alignment, the multiplication results of POSIT8 data have fewer overlap bits than FP8 data. In particular, 28.4% of the dynamically aligned results of POSIT8 data even have no overlap bits, as shown in Fig. 11. When accumulating two operands with no overlap bits, we can substitute the adders to simple OR logic without affecting the output as all adders perform X + 0, which equals to X|0. The OR-based accumulation can save energy by 82.9% than the adders, inspiring us to substitute more adders with OR logic. We propose a CASU with early detection and cyclical scheduling mechanisms, which allows replacing adders with OR gates even if the operands have overlap bits, thereby reducing energy for the adder tree.

First, apart from the conditions of operands with no overlap bit, we propose an early detection approach for the operands with overlap bits to detect whether the specific overlap bits are both "1." As shown in Fig. 11, the 4-bit operands of A and B may have two overlap bits, which are $A[1]/B[3]$ and $A[0]/B[2]$. As we can see, the result of $A[1] + B[3]$ also equals to $A[1]|B[3]$, if $A[1]$ and $B[3]$ are not both "1," defined as a substituting condition. The substituting probability is 3/4. Once all overlap bits satisfy the substituting condition, we can still compute $A + B$ through $A|B$. Intuitively, the more bits we detect, the higher the substituting ratio achieves, leading to more energy savings. However, the energy reduction suffers from a bottleneck for two reasons. First, the probability of substituting adders with OR decreases exponentially as the number of overlapping bits increases. For n overlap bits, simultaneously satisfying the substituting condition leads to a $(3/4)^n$ probability, which is only 23.7%. Second, the detection overhead increases linearly with the number of detected bits, limiting the energy efficiency. According to our experimental results, an overlap bit detection count of 4 leads to optimal energy efficiency, as shown in Fig. 11. The early detection reduces 36.1% of adders for operands with overlap bits.

Second, if two operands have overlap bits that are both "1," we permute the computing with a cyclical scheduling, which

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

creates no overlap conditions for operands with overlap bits. Theoretically, if two mantissa multiplication results, defined as $M_0 = W_0 \times A_0$ and $M_1 = W_1 \times A_1$, have overlap bits after the alignment, it has to use adders for computing. Instead of directly computing $M_0 + M_1$ with adders, CASU permutes the bit-serial computing of M_0 and M_1 to create no overlap conditions. Specifically, assuming $M_0 = W_0 \times A_0$ and $M_1 = W_1 \times A_1$ have 1 overlap bit, it indicates that all of the bit-serially computed $W_0 \times A_0[i]$ and $W_1 \times A_1[i]$ have 1 overlap bit, as shown in Fig. 12. Therefore, every $W_0 \times A_0[i] + W_1 \times A_1[i]$ requires an adder. Rather than synchronously computing $A_0[i]$ and $A_1[i]$, the scheduler cyclically shifts A_0 by 1-bit, which permutes the bit-serial computing of A_0 from $A_0[0{:}n]$ to $A_0[1{:}n]$ and then $A_0[0]$. In this way, the CIM array computes $A_0[i+1]$ and $A_1[i]$ together, where $W_0 \times A_0[i+1]$ and $W_1 \times A_1[i]$ have no overlap bit if $i < n$, as shown in Fig. 12. The overlap condition only happens in $W_0 \times A_0[0] + W_1 \times A_1[n]$, which uses 1 adder. As a result, CASU uses $(n-1)$ OR with 1 adder for completing n additions. The cyclical alternating scheduling approach increases a 36.4% no overlap ratio for a 31.1% energy reduction.

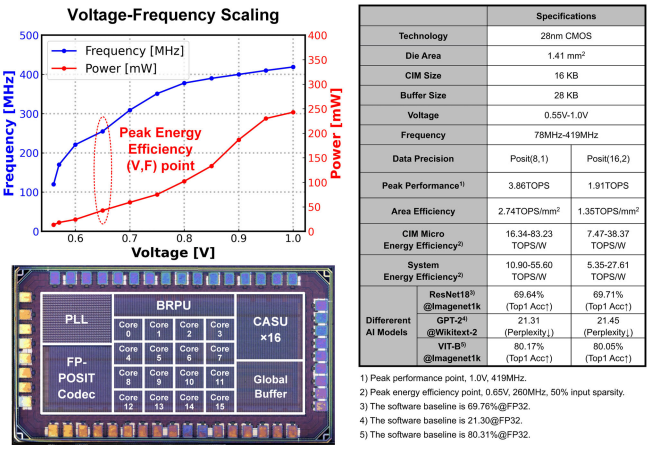
B. CASU for OR-Based Accumulation

Fig. 13 shows the overall architecture of CASU, comprising two alignment units, a work mode controller, and a cyclical computing scheduler. During computing, the alignment unit receives two decimal regime from BRPU, defined as R_D^0 and R_D^1, corresponding to the mantissa multiplications of $W_0 \times A_0$ and $W_1 \times A_1$, respectively. It compares R_D^0 with R_D^1 and computes $|R_D^0 - R_D^1|$. Assuming $R_D^0 > R_D^1$, the alignment unit will right shift $W_1 \times A_1$ by $|R_D^0 - R_D^1|$ bits to align to $W_0 \times A_0$ for accumulation. Based on $|R_D^0 - R_D^1|$ and the bit-width of the mantissa multiplication results, defined as n, the work mode controller obtains the overlap bit counts, which is $n - |R_D^0 - R_D^1|$. It then determines whether to use the OR mode or the adder mode with cyclical scheduling with $n - |R_D^0 - R_D^1|$. Fig. 13 shows the flowchart of the controller. Specifically, if $n - |R_D^0 - R_D^1| \le 0$, denoting no overlap bit, the controller selects OR mode for accumulation. Else, if $n - |R_D^0 - R_D^1| > 4$, it uses adder mode with cyclical scheduling. Otherwise, if $0 < n - |R_D^0 - R_D^1| < 4$, the controller checks whether all overlap bits satisfy the substitute condition, indicating that none of the overlap bits are both "1." It uses the OR mode if meeting the substitute condition while utilizing the adder mode with cyclical scheduling if not satisfying the substitute condition. When working in the adder mode with cyclical scheduling, the scheduler permutes the bit-serial computing of $W_0 \times A_0$ and $W_1 \times A_1$. It cyclically shifts the bit-serial computing of $W_0 \times A_0$ by $n - |R_D^0 - R_D^1|$ bits, which is asynchronous with $W_1 \times A_1$. It avoids the overlap bits in synchronous computing to improve the ratio of using OR logic to substitute adders for energy saving.

VII. Measurement Results

A. Chip Specification and Verification System Setup

Fig. 14 presents the die photograph and performance summary of the proposed PD-CIM macro, fabricated in 28 nm



		Specifications	
Technology		28nm CMOS	
Die Area		1.41 mm²	
CIM Size		16 KB	
Buffer Size		28 KB	
Voltage		0.55V-1.0V	
Frequency		78MHz-419MHz	
Data Precision		Posit(8,1)	Posit(16,2)
Peak Performance[1]		3.86TOPS	1.91TOPS
Area Efficiency		2.74TOPS/mm²	1.35TOPS/mm²
CIM Micro Energy Efficiency[2]		16.34-83.23 TOPS/W	7.47-38.37 TOPS/W
System Energy Efficiency[2]		10.90-55.60 TOPS/W	5.35-27.61 TOPS/W
Different AI Models	ResNet18[3] @Imagenet1k	69.64% (Top1 Acc↑)	69.71% (Top1 Acc↑)
	GPT-2[4] @Wikitext-2	21.31 (Perplexity↓)	21.45 (Perplexity↓)
	VIT-B[5] @Imagenet1k	80.17% (Top1 Acc↑)	80.05% (Top1 Acc↑)

1) Peak performance point, 1.0V, 419MHz.
2) Peak energy efficiency point, 0.65V, 260MHz, 50% input sparsity.
3) The software baseline is 69.76%@FP32.
4) The software baseline is 21.30@FP32.
5) The software baseline is 80.31%@FP32.

Fig. 14. Chip micrograph and summary of the proposed PD-CIM.

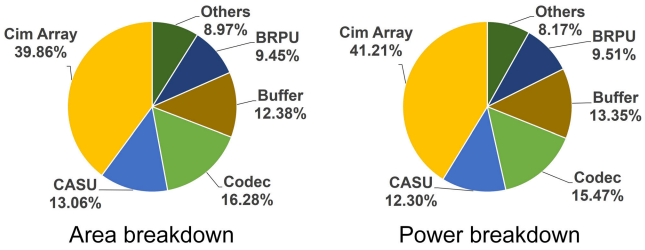


Fig. 15. Power and area breakdown of the proposed PD-CIM.

CMOS technology with a 1.41 mm^2 area. The proposed PD-CIM supports POSIT8 and POSIT16-based MAC computation, which equals two operations. Under the supply voltage of 0.55-to-1.0 V, the PD-CIM consumes a power of 5.5-to-237 mW. The maximum frequency of this processor is 419 MHz, resulting in a peak performance of 3.86 TOPS for POSIT8, which is 1.91 TOPS for POSIT16. Specifically, PD-CIM contains a 16 KB CIM array. Each cycle, PD-CIM activates 8 KB CIM cells for computing, indicating $8 \times 1024 \times 8.4 = 16\,384$ 4-bit CIM units. For POSIT8 data, the average mantissa bit-width is 3.55 bits. Thus, it consumes an average of 3.55 cycles for each POSIT8 computing. When operating at a frequency of 419 MHz, the peak performance is $16\,384 \times 419 \times 2.3.55/10^6 = 3.86$ TOPS, where 2 indicates 1 multiplication and 1 addition in each cycle. With the proposed optimizations, PD-CIM reaches a peak energy efficiency of 83.23 TFLOPS/W for POSIT(8,1) achieved at 0.65 V with 260 MHz frequency.

Fig. 15 demonstrates the area and power breakdown of the PD-CIM. The shift-OR-based BPPU takes up an area ratio of 9.45%, which is 67.8% smaller than the conventional codec-adder-based regime processing. Besides, the simplified shift-OR logic facilitates the power proportion of BPPU to reduce to 9.51%. The CPCS CIM array reduces the power ratio from 56.3% to 41.21% since it pre-computes the critical bits and stores them in spare CIM cells, saving computing power. The CASU significantly decreases the power ratio of the adder tree from 23.4% to 12.3% by substituting the adders with OR logic. The area overhead is only a negligible 2.3% since CASU only increases a small number of OR gates.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

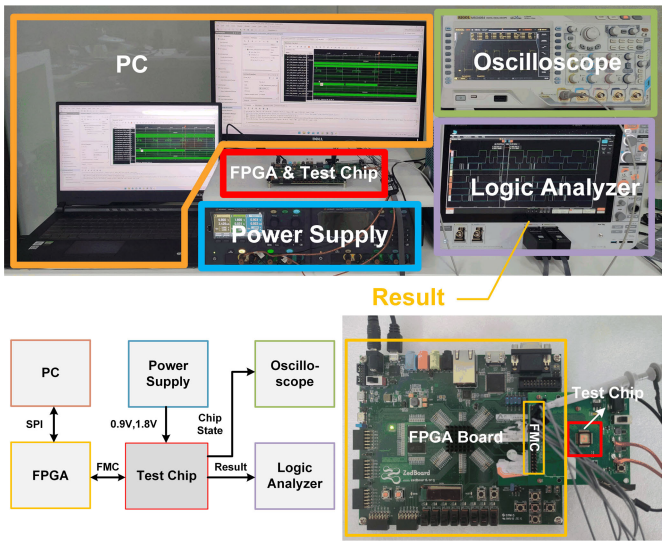


Fig. 16. Verification system of the proposed PD-CIM.

TABLE I
COMBINED EVALUATION ON THREE TYPICAL AI MODELS

Models	ResNet18	GPT-2	ViT-B
Dataset	ImageNet-1K	Wikitext-2	ImageNet-1K
Task	Training	Inference	Inference
Data Precision	POSIT(16,2)	POSIT(8,1)	POSIT(8,1)
Accuracy	69.7%	21.31	80.17%
Accuracy Loss[1)]	-0.04%	-0.05%	-0.14%
Performance(TOPS)[1)]	1.73	2.95	2.91
CIM Macro Energy Efficiency[2)] (TFLOPS/W)	16.27	34.30	33.81
System Energy Efficiency[2)] (TFLOPS/W)	10.45	21.79	21.48
Energy Saving[3)]	8.81×	7.90×	7.12×

1) Compared with training or inference with FP32 data precision.
2) Measured at 0.65V with 260MHz for peak energy efficiency.
3) Including on-chip components while excluding off-chip data access.

1) Measurement: Fig. 16 illustrates the verification system for the fabricated PD-CIM macro, consisting of a PD-CIM test chip, a Xilinx VC709 FPGA board, a dc power, an oscilloscope, a logic analyzer, and a host computer. PD-CIM communicates with the FPGA through an integrated FPGA mezzanine connector (FMC). It loads the weight parameters to the CIM array and stores the activation in the global buffer. During data loading, PD-CIM transfers the FP operands to a specific POSIT data format determined by the analyzed distribution from the host computer. After computation, PD-CIM outputs run flag signals and the results to the FPGA for workflow scheduling.

2) Simulation: Besides evaluating the performance of a single PD-CIM macro individually, we also simulated the systematic performance of multiple integrated PD-CIM macros. Specifically, we connected 16 PD-CIMs to an LPDDR3-800 via a bus, where the bandwidth is 6.4 GB/s. The 16 PD-CIM macros are assigned tasks in a tensor-parallel manner, with each PD-CIM macro independently performing tensor computations. Compared to a single PD-CIM macro, the multiple PD-CIM system achieves nearly linear performance improvement, which is 15.3× than that of a single CIM macro.

B. Evaluation on Key Features

Figs. 17 and 18 detail the evaluation results of the proposed techniques on three typical models, where the PD-CIM works at the peak energy efficiency VF point of 0.65 V under 260 MHz. The energy efficiency of PD-CIM for ResNet18 training with POSIT16 data format is 5.35 TFLOPS/W, which reduces to 3.48 TFLOPS/W when including the global SRAM access while excluding the off-chip DRAM access.

1) BRPU Evaluation: Fig. 17 shows the energy efficiency improvement achieved with BRPU. By simplifying the complicated codec-addition computing logic of the regime processing to the shift-OR logic operations, BRPU increases the energy efficiency by an average of 29.7% for the three AI models. In addition, the bi-directional shift scheduling further improves energy efficiency by 39%. For ResNet18 training, BRPU increases the peak energy efficiency from 3.48 to 6.75 TFLOPS/W for POSIT16 with an improvement ratio of 1.94×, as shown in Fig. 18. Besides, it boots the peak energy efficiency of CIM core to 27.94 TFLOPS/W for POSIT8.

2) CPCS Evaluation: Fig. 17 also presents the throughput improvement with CPCS. Instead of wasting CIM cells caused by mantissas with dynamic bit-width, CPCS utilizes the spare CIM cells to store the pre-computed partial summation results, enabling dual-bit MACs in each cycle. This pre-compute-and-store mechanism increases the throughput of the CIM array by an average of 38.2% when storing the 3-bit weight parameters with 4-bit CIM units. In addition, the CPCS CIM array allows a 2–4-bit weight association, providing an extra throughput improvement of 24.7%. Moreover, the CPCS CIM array can perform a 3-bit mode priority partition for the POSIT16 data format, increasing the counts of CIM units working at dual-bit MACs for high throughput. As shown in Fig. 18, CPCS increases the systemic energy efficiency by an average of 1.46×, which is up to 1.96×, resulting in a peak value of 13.23 TFLOPS/W at POSIT16. It improves the peak energy efficiency of the CIM core to 45.54 TFLOPS/W at POSIT8.

3) CASU Evaluation: Fig. 17 details the energy reduction achieved by CASU. For all models, CASU saves the energy of the adder tree by substituting additions to the OR logic once the dynamically aligned mantissas have no overlap bits. Since the power of OR logic is 82.9% smaller than addition, the bit-wise OR computing of CASU reduces an average of 25.7%. In addition, CASU contains a cyclically alternating scheduler that increases the ratio of using OR logic to replace additions for power saving. The cyclical scheduling of CASU further reduces 31.3% of computing power. With CASU, the systemic energy efficiency increases to 27.61 TFLOPS/W for POSIT16, achieving 2.08× improvement. In addition, the peak energy efficiency of the CIM core reaches 83.23 TFLOPS/W at POSIT8.

4) Combined Evaluation: Table I illustrates the combined evaluation results of the three typical models, including training and inference. Compared with training ResNet18 using

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

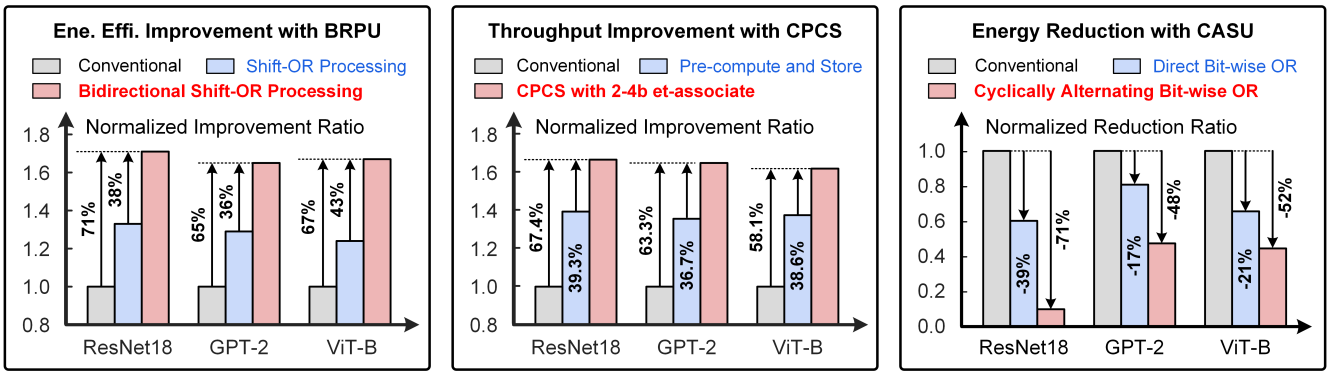


Fig. 17. Performance evaluations on the three key features.

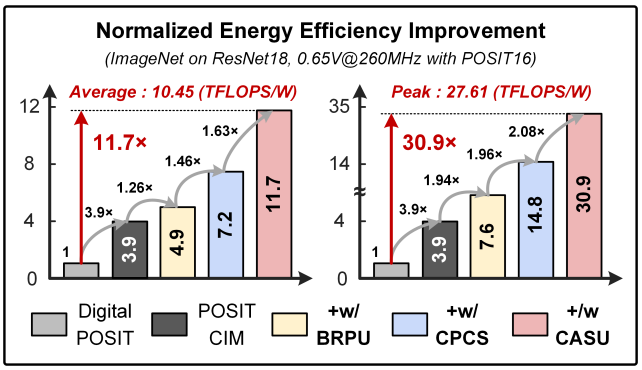


Fig. 18. Energy efficiency improvement with combined evaluation.

FP32 data, POSIT16 only suffers from 0.04% accuracy loss. The inference accuracy of ViT-B is 0.14% lower than FP32 since the softmax function in attention computing is sensitive to the data precision. Although there is a slight loss compared to FP32, the accuracy of POSIT8 data is almost identical to FP16, surpassing FP8 by 4.76%. The reason is that POSIT data dynamically allocates bits to adapt to the data distribution of attention computing. For the three models, PD-CIM saves energy by 7.94×. The energy efficiency of GPT-2 and ViT-B for inference exceeds ResNet18 training for two reasons. First, the training process uses a POSIT16 data format, consuming 2× memory access energy. Second, the throughput of PD-CIM for POSIT16 computation is half that of POSIT8.

Table II compares the time and energy consumption for the ViT-B model on ImageNet classification among FP16-based CIM, digital POSIT8, POSIT8-based CIM without optimization, and the proposed PD-CIM. For FP16-based CIM and digital POSIT8, all 12 layers in the ViT-B model consume identical time and energy. As shown in Table II, even directly using digital POSIT8 computing can save 48.4% of the time and reduce energy by 38.9% compared to the FP16-based CIM since the activation access and MAC power of POSIT8 is nearly half the size of FP16. The energy reduction is less than 50%, coming from the reason that FP16-based CIM can avoid the weight parameter access for energy saving. The POSIT8-based CIM can increase the speed by 37.9% and save energy by 19.0% compared to digital POSIT8 computing. However, the dynamic property of the POSIT data format, which introduces a significant overhead of regime processing, CIM cell underutilization, and energy waste of the adder tree, limits the energy efficiency of POSIT8-based CIM. The proposed PD-CIM effectively handles the dynamic bit allocation of POSIT data, breaking the energy efficiency of POSIT8-based CIM. It reduces the regime processing overhead by simplifying the complicated codec logic to shift-OR operations, improves the CIM cell utilization by using the spare CIM cells for dual-bit MACs, and saves the energy of the adder tree by substituting the additions to the OR logic. As a result, the proposed PD-CIM achieves 1.98× speedup and 3.35× energy reduction than POSIT8-based CIM without optimizations.

TABLE II
LATENCY AND ENERGY COMPARISONS WITH DIFFERENT SETUPS

	Latency[1] (ms)			Energy[1] (mJ)		
FP16 CIM	6.49× 12[2]			0.95× 12[2]		
Digital POSIT8	3.35× 12[2]			0.58× 12[2]		
Layers	**Trad.[3]**	**Ours.[4]**	**Imp.[5]**	**Trad.[3]**	**Ours.[4]**	**Imp.[5]**
Layer_1	1.79	0.86	**2.09×**	0.44	0.12	**3.70×**
Layer_2	2.10	1.04	**2.02×**	0.46	0.13	**3.42×**
Layer_3	2.00	1.06	**1.89×**	0.47	0.14	**3.25×**
Layer_4	2.11	1.08	**1.95×**	0.46	0.14	**3.70×**
Layer_5	2.18	1.07	**2.03×**	0.50	0.15	**3.39×**
Layer_6	2.13	1.10	**1.93×**	0.49	0.14	**3.45×**
Layer_7	2.07	1.08	**1.93×**	0.49	0.15	**3.37×**
Layer_8	2.05	1.03	**1.99×**	0.48	0.14	**3.38×**
Layer_9	2.19	1.06	**2.06×**	0.47	0.16	**3.05×**
Layer_10	2.16	1.13	**1.92×**	0.47	0.15	**3.19×**
Layer_11	2.13	1.07	**1.99×**	0.47	0.15	**3.15×**
Layer_12	2.07	1.06	**1.95×**	0.48	0.15	**3.14×**

1) Including global SRAM access while excluding DRAM access.
2) Identical latency and energy for all 12 layers.
3) POSIT-based CIM without optimizations.
4) POSIT-based CIM with the proposed optimizations.
5) Improvement of with optimizations than without optimizations.

C. Comparison With State-of-the-Art Processors

Table III compares PD-CIM with previous state-of-the-art FP-CIMs. Under the same training or inference accuracy, the proposed PD-CIM achieves the highest energy efficiency of 83.23 TFLOPS/W. The energy efficiency outperforms [16] by 2.6× when evaluated with the same 50% of sparsity ratio. It comes from three reasons. First, the BRPU avoids

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

TABLE III
MEASUREMENT RESULTS AND PERFORMANCES COMPARISON WITH STATE-OF-THE-ART FP-CIM PROCESSORS

	ISSCC'22 [25]	ISSCC'23 [15]	ISSCC'23 [16]	ISSCC'23 [17]	ISSCC'21 [27]	This Work
Dynamic Format	NO	NO	NO	NO	YES	**YES**
Technology (nm)	28	22	28	28	28	**28**
Area (mm^2)	6.69	18	0.146	4.54	3.8	**1.41**
Supply Voltage (V)	0.6-1.0	0.6-0.8	0.6-0.9	0.397-0.9	0.6-0.9	**0.55-1.0**
Frequency (MHz)	50-220	NA	NA	10-400	104-288	**78-419**
Precision	FP32/BF16 INT16/INT8	BF16	BF16/INT8	FP16/BF16 INT8/INT4	32b/16b/8b CUSTOM POSIT	**POSIT16** **POSIT8**
Power (mW)	12.5-69.4	NA	NA	0.87-74.9	50-230	**5.5-237**
Peak Performance (TFLOPS)	0.14@FP32 1.35@INT8	1.24-1.28	NA	1.64-9.63@INT4[3)]	0.0163@POSIT16 0.0337@POSIT8	**1.91@POSIT16** **3.86@POSIT8**
Energy Efficiency (TFLOPS/W)[4)5)]	3.7@FP32 36.5@INT8	16.2-70.2[1)]	14-31.6@BF16[2)] 19.5-44@INT8[2)]	3.2-16.9@FP16[3)] 51-300@INT4[3)]	0.121@POSIT16 0.248@POSIT8	**38.37@POSIT16[2)]** **83.23@POSIT8[2)]**
Area Efficiency (TFLOPS/mm^2)	0.02@FP32 0.20@INT8	0.069-0.071[1)]	NA	0.36-2.12@INT4[3)]	0.0043@POSIT16 0.0089@POSIT8	**1.35@POSIT16[2)]** **2.74@POSIT8[2)]**

1) Evaluated with 90% input sparsity. 2) Evaluated with 50% input sparsity. 3) Average performance from dense models to sparse models.
4) Including on-chip SRAM data access energy. 5) Excluding off-chip memory data access energy.

complicated codec logic of regime processing to save power. Second, the CPCS CIM array increases the throughput by utilizing the spare CIM cell for dual-bit MACs in each cycle. Third, CASU uses bit-wise or operation to reduce the energy of the adder tree. Besides, the average energy efficiency of PD-CIM, which is 41.5 TFLOPS/W at POSIT8, surpasses [15] by 2.36× and exceeds [17] by 2.46×. On the one hand, PD-CIM uses 8-bit data instead of 16-bit data for high-accuracy applications, saving 50% of data access energy. On the other hand, the proposed BRPU reduces the regime processing overhead by 67.7%, and CASU saves 57% of the energy consumed by the adder tree. Moreover, PD-CIM reaches 3.51× speedup than [15] for two reasons. First, the BRPU and CASU simplify the computational complexity of regime processing and adder tree, releasing the critical path for a high frequency of 419 MHz, which is 2.68× higher than that of [15]. Second, instead of using bit-serial computing, the CPCS CIM array can perform dual-bit MACs with a pre-compute-and-store mechanism, increasing the throughput by 1.31×.

VIII. DISCUSSION

This section discusses and analyzes the applicability of the proposed technique to traditional FP data formats and classical POSIT-based DNN processors.

A. Applicability to Standard FP Data

The proposed CPCS and CASU are applicable to standard FP data. Specifically, for conventional FP16, CPCS regards them as POSIT16 data with a fixed mantissa bit-width of 10 bits. It uses the 3-bit priority partition mechanism to separate the 10-bit mantissa into three segments. Then, it performs dual-bit MACs with the proposed pre-compute-and-store approach to improve the throughput of mantissa multiplication. In addition, once the dynamically aligned mantissas have overlap bits that are less than 4 bits, CASU works to replace the original additions with OR logic operations to reduce energy from the adder tree.

B. Applicability to Classical POSIT-Based DNN Processors

Besides being used in CIM processors, the proposed BRPU and CASU can apply to classical POSIT-based DNN processors. Specifically, BRPU can significantly reduce the overhead of regime processing, which increases considerable area and power consumption for all POSIT-based processors. Compared with BRPU in classical POSIT-based DNN processors, BRPU in CIM shows more evident advantages. The reason is that the CIM processor significantly reduces MAC's computation area and power consumption, making the proportion of regime processing overhead more prominent. For classical POSIT-based DNN processors, CASU can also replace addition with OR to reduce the energy for accumulation. The energy gain is relatively minor compared to using CASU in CIM. CASU changes the bit-serial computing order of CIM to increase the proportion of replacing addition with OR logic, but the classical POSIT-based DNN processors prefer parallel multiplication.

IX. CONCLUSION

This article proposes a POSIT-based CIM macro that significantly improves the energy efficiency of prior FP-CIMs based on conventional FP data format while maintaining application accuracy. The superior energy efficiency performance arises from the proposed three approaches, which effectively handle the challenges caused by the dynamic properties of the POSIT data format. First, a BRPU simplifies the codec and addition in regime processing to shift-OR-based logic operations, significantly reducing the overhead from dynamic regime processing. Second, a CPCS CIM array adopts a pre-compute-and-store mechanism to utilize the spare CIM cells caused by dynamic bit-width mantissas, enabling dual-bit MACs in the CIM array to improve CIM cell utilization. Third, a CASU uses bit-wise OR logic operations to substitute the data-wise addition once the two dynamically aligned mantissas have no overlap bits, remarkably saving the energy in the adder tree.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Z. Liu et al., "BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 2774–2781.

[2] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 4172–4182.

[3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 10674–10685.

[4] G. Metzer, E. Richardson, O. Patashnik, R. Giryes, and D. Cohen-Or, "Latent-NeRF for shape-guided generation of 3D shapes and textures," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 12663–12673.

[5] D. Han, J. Ryu, S. Kim, S. Kim, and H.-J. Yoo, "MetaVRain: A 133 mW real-time hyper-realistic 3D-NeRF processor with 1D-2D hybrid-neural engines for metaverse on mobile devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 50–52.

[6] J. Ryu et al., "Neugpu: A 18.5mJ/iter neural-graphics processing unit for instant-modeling and real-time rendering with segmented-hashing architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 372–374.

[7] R. Guo et al., "A 28 nm 74.34TFLOPS/W BF16 heterogenous CIM-based accelerator exploiting denoising-similarity for diffusion models," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 362–364.

[8] Y. Wang et al., "Trainer: An energy-efficient edge-device training processor supporting dynamic weight pruning," *IEEE J. Solid-State Circuits*, vol. 57, no. 10, pp. 3164–3178, Oct. 2022.

[9] S. Kim et al., "C-DNN: An energy-efficient complementary deep-neural-network processor with heterogeneous CNN/SNN core architecture," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 157–172, Jan. 2024.

[10] S. K. Venkataramanaiah et al., "A 28-nm 8-bit floating-point tensor core-based programmable CNN training processor with dynamic structured sparsity," *IEEE J. Solid-State Circuits*, vol. 58, no. 7, pp. 1885–1897, Jul. 2023.

[11] J. Park, S. Lee, and D. Jeon, "A neural network training processor with 8-bit shared exponent bias floating point and multiple-way fused multiply-add trees," *IEEE J. Solid-State Circuits*, vol. 57, no. 3, pp. 965–977, Mar. 2022.

[12] Z.-S. Fu, Y.-C. Lee, A. Park, and C.-H. Yang, "A 40-nm 646.6TOPS/W sparsity-scaling DNN processor for on-device training," in *Proc. IEEE Symp. VLSI Technol. Circuits*, Jun. 2022, pp. 40–41.

[13] D. Han et al., "HNPU: An adaptive DNN training processor utilizing stochastic dynamic fixed-point and active bit-precision searching," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2858–2869, Sep. 2021.

[14] J. Lee et al., "A 13.7 TFLOPS/W floating-point DNN processor using heterogeneous computing architecture with exponent-computing-in-memory," in *Proc. Symp. VLSI Circuits*, Jun. 2021, pp. 1–2.

[15] P.-C. Wu et al., "A 22 nm 832Kb hybrid-domain floating-point SRAM in-memory-compute macro with 16.2–70.2TFLOPS/W for high-accuracy AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 126–128.

[16] A. Guo et al., "A 28 nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 128–130.

[17] J. Yue et al., "A 28 nm 16.9–300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 1–3.

[18] S. Yan et al., "A 28-nm floating-point computing-in-memory processor using intensive-CIM sparse-digital architecture," *IEEE J. Solid-State Circuits*, vol. 59, no. 8, pp. 2630–2643, Aug. 2024.

[19] X. Chen et al., "Toggle rate aware quantization model based on digital floating-point computing-in-memory architecture," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 6, pp. 3181–3185, Jun. 2024.

[20] X. Hu et al., "An 8.8 TFLOPS/W floating-point RRAM-based compute-in-memory macro using low latency triangle-style mantissa multiplication," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 11, pp. 4216–4220, Nov. 2023.

[21] P.-C. Wu et al., "A floating-point 6T SRAM in-memory-compute macro using hybrid-domain structure for advanced AI edge chips," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 196–207, Jan. 2024.

[22] Y. Wang et al., "A 28 nm 83.23TFLOPS/W POSIT-based compute-in-memory macro for high-accuracy AI applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 566–568.

[23] W.-S. Khwa et al., "A 16 nm 96 Kb integer/floating-point dual-mode-gain-cell-computing-in-memory macro achieving 73.3–163.3TOPS/W and 33.2–91.2TFLOPS/W for AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 568–570.

[24] Y. Yuan et al., "A 28 nm 72.12TFLOPS/W hybrid-domain outer-product based floating-point sram computing-in-memory macro with logarithm bit-width residual ADC," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 576–578.

[25] F. Tu et al., "A 28 nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, Feb. 2022, pp. 1–3.

[26] T.-H. Wen et al., "A 22 nm 16Mb floating-point ReRAM compute-in-memory macro with 31.2TFLOPS/W for AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 580–582.

[27] N. Shah, L. I. G. Olascoaga, S. Zhao, W. Meert, and M. Verhelst, "PIU: A 248 GOPS/W stream-based processor for irregular probabilistic inference networks using precision-scalable posit arithmetic in 28 nm," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 150–152.

[28] Y. Wang, D. Deng, L. Liu, S. Wei, and S. Yin, "PL-NPU: An energy-efficient edge-device DNN training processor with posit-based logarithm-domain computing," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 10, pp. 4042–4055, Oct. 2022.

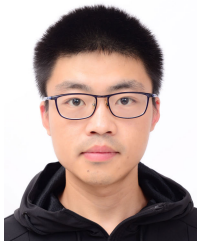
[29] R. Murillo, A. A. D. Barrio, G. Botella, and C. Pilato, "Generating posit-based accelerators with high-level synthesis," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 10, pp. 4040–4052, Oct. 2023.

[30] H. Zhang and S.-B. Ko, "Variable-precision approximate floating-point multiplier for efficient deep learning computation," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 5, pp. 2503–2507, May 2022.

[31] Q. Li, C. Fang, and Z. Wang, "PDPU: An open-source posit dot-product unit for deep learning applications," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2023, pp. 1–5.

[32] M. Zolfagharinejad, M. Kamal, A. Afzali-Khusha, and M. Pedram, "Posit process element for using in energy-efficient DNN accelerators," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 6, pp. 844–848, Jun. 2022.

[33] N. Shah, L. I. G. Olascoaga, S. Zhao, W. Meert, and M. Verhelst, "DPU: DAG processing unit for irregular graphs with precision-scalable posit arithmetic in 28 nm," *IEEE J. Solid-State Circuits*, vol. 57, no. 8, pp. 2586–2596, Aug. 2022.



Yang Wang received the B.S. degree in electronic science and technology from Xidian University, Xi'an, China, in 2014, and the Ph.D. degree in microelectronics and solid state electronics from the Institute of Microelectronics, Chinese Academy of Sciences, Beijing, China, in 2019.

He is currently an Associate Researcher with the School of Integrated Circuits, Tsinghua University, Beijing. His research interests include very-large-scale integration digital signal processing (VLSI DSP), deep learning, and neural network acceleration.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.



Xiaolong Yang received the B.S. degree in micro-electronic science and engineering from Tsinghua University, Beijing, China, in 2021, where he is currently pursuing the M.S. degree with the School of Integrated Circuits.

His current research interests include deep learning, computer architecture, and very-large-scale integration (VLSI) design.



Yubin Qin received the B.S. degree from the School of Electronic Science and Engineering, Southeast University, Nanjing, China, in 2020. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits, Tsinghua University, Beijing, China.

His current research interests include deep learning, very-large-scale integration (VLSI) design, and hardware-software co-design.



Zhiren Zhao received the B.S. degree from the College of Electronic Science and Engineering, Jilin University, Changchun, China, in 2021. He is currently pursuing the M.S. degree with the School of Integrated Circuits, Tsinghua University, Beijing, China.

His current research interests include neural network acceleration, very-large-scale integration (VLSI) design, and hardware-software co-design.



Ruiqi Guo received the B.S. degree from the School of Microelectronics, Xidian University, Xi'an, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits, Tsinghua University, Beijing, China.

His current research interests include static random access memory (SRAM)-based computing in memory, computer architecture and very-large-scale integration (VLSI) design, and hardware-software co-design for artificial intelligence (AI) accelerators.



Zhiheng Yue received the B.S. degree in electronic science and technology from Beijing University of Posts and Telecommunications, Beijing, China, in 2017, and the M.S. degree in electrical and computer engineering from the University of Michigan, Ann Arbor, MI, USA, in 2019. He is currently pursuing the Ph.D. degree in electronic science and technology with Tsinghua University, Beijing.

His current research interests include deep learning, computation-in-memory, artificial intelligence (AI) accelerators, and very-large-scale-integration (VLSI) design.



Huiming Han received the B.S. degree from the School of Electronic and Information Engineering, Hebei University of Technology, Tianjin, China, in 2005.

He is currently a Senior Engineer with Tsinghua University, Beijing, China. His research interests include circuit design, compilation optimization, and intelligent computing chip systems.



Shaojun Wei (Fellow, IEEE) was born in Beijing, China, in 1958. He received the Ph.D. degree from Faculte Polytechnique de Mons, Mons, Belgium, in 1991.

He became a Professor at the Institute of Microelectronics, Tsinghua University, Beijing, in 1995. His main research interests include very-large-scale integration system-on-chip (VLSI SoC) design, electronic design automation (EDA) methodology, and communication application-specific integrated circuit (ASIC) design.

Dr. Wei is a Senior Member of the Chinese Institute of Electronics (CIE).



Yang Hu (Member, IEEE) received the B.S. degree from Tianjin University, Tianjin, China, in 2007, the M.S. degree from Tsinghua University, Beijing, China, in 2011, and the Ph.D. degree from the University of Florida, Gainesville, FL, USA, in 2017.

He was an Assistant Professor with the ECE Department, The University of Texas at Dallas, Richardson, TX, USA, from 2017 to 2021. He is currently an Associate Professor with the School of Integrated Circuits, Tsinghua University.



Shouyi Yin (Fellow, IEEE) received the B.S., M.S., and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2000, 2002, and 2005, respectively.

He was a Research Associate with Imperial College London, London, U.K. He is currently a Full Professor and the Vice-Director at the School of Integrated Circuits, Tsinghua University. He has published more than 100 journal articles and more than 50 conference papers. His research interests include reconfigurable computing, AI processors, and high-level synthesis.

Dr. Yin has served as a Technical Program Committee Member for the top very-large-scale integration (VLSI) and electronic design automation (EDA) conferences, such as Asian Solid-State Circuits Conference (A-SSCC), IEEE/ACM International Symposium on Microarchitecture (MICRO), Design Automation Conference (DAC), International Conference on Computer-Aided Design (ICCAD), and Asia and South Pacific Design Automation Conference (ASP-DAC). He is also an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS (TCAS-I), ACM *Transactions on Reconfigurable Technology and Systems* (TRETS), and *Integration*, the VLSI Journal.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:24:32 UTC from IEEE Xplore. Restrictions apply.